

PROJECT DOCUMENTATION

ON

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT) IN SERVICENOW

Submitted in partial fulfillment of the requirements for the Naan Mudhalvan Skill Development Programme



NAAN MUDHALVAN
Skill Development Programme
Submitted By
Team ID: SWTID-2026-6763

S.No.	Team Member Name	Role
1	R. Sri Nandha Bala	Project Developer (Client Scripts)
2	R. Yokeswaran	UI Policy Configuration
3	V. Sriram	Testing and Validation
4	S. Sri Haran	Documentation and Project Coordination
5	D. Thamizh	Project Planning and Architecture

Submitted To

B.Tech – Artificial Intelligence and Data Science

SRM MADURAI COLLEGE FOR ENGINEERING AND TECHNOLOGY

ANNA UNIVERSITY

INDEX

1. Introduction	4
Project Title.....	4
Team Members	4
2. Project Overview.....	4
Purpose	4
Features.....	4
3. Architecture	5
System Architecture.....	5
Platform	5
Client Scripts and UI Policies.....	5
Database.....	5
4. Setup Instructions.....	5
Prerequisites.....	5
Installation and Configuration	6
Step 1: Access ServiceNow Developer Instance	6
Step 2: Create the Client Scripts	6
Step 3: Create the UI Policies	6
Step 4: Sample Client Script.....	6
Step 5: Test the Configuration	6
5. Folder Structure.....	6
Logical Project Structure	7
6. Running the Application	7
Steps to Run the Project.....	7
Expected Output	8
7. API Documentation.....	8
API Integration	8
Core Operations	8
Optional REST API Integration.....	8
8. Authentication	8
Authentication.....	8
Authorization	9

Security	9
9. User Interface	9
Main Interface Components.....	9
Screenshots	9
10. Testing.....	10
Testing Types	10
User Acceptance Testing (UAT)	10
Test Results	10
11. Screenshots or Demo.....	11
Demonstration Steps.....	11
Demo Video	11
GitHub Repository.....	11
12. Known Issues	11
13. Future Enhancements	12
Conclusion.....	12

1. Introduction

Project Title

Implement Client Script & UI Policy (Incident) in ServiceNow

Team Members

S.No.	Team Member Name	Role
1	R. Sri Nandha Bala	Project Developer (Client Scripts)
2	R. Yokeswaran	UI Policy Configuration
3	V. Sriram	Testing and Validation
4	S. Sri Haran	Documentation and Project Coordination
5	D. Thamizh	Project Planning and Architecture

2. Project Overview

Purpose

The purpose of this project is to improve data quality and user guidance on the ServiceNow Incident form using Client Scripts and UI Policies. In many service desks, incidents are raised with missing or unclear details, and closure information is entered inconsistently. This leads to delays, follow-ups and rework for support teams.

This project addresses these challenges by validating user input and controlling field behaviour directly on the form, in the user's browser, before the record is saved.

Features

- onLoad Client Script: Displays guidance when a new incident form opens.
- onChange Client Script: Reacts to field changes, for example alerting the user when Priority is set to Critical.
- onSubmit Client Script: Validates the short description and blocks invalid submissions.
- UI Policy - Mandatory fields: Makes Close notes and Close code mandatory when State is Resolved.
- UI Policy - Conditional fields: Makes Configuration item mandatory for the Network category.
- UI Policy - Read-only fields: Locks selected fields when State is Closed.
- Instant feedback: All checks run client-side without a server round trip.
- Testing and Validation: Behaviour verified through functional, performance and user acceptance testing.

3. Architecture

System Architecture

The project is developed entirely within the ServiceNow platform. Client Scripts and UI Policies are configured on the Incident table and execute in the user's browser when the form loads, when fields change and when the form is submitted. No separate frontend or backend application is required.

The system consists of the following components:

- Employee / End User: Opens the Incident form and enters issue details.
- Incident Form: The standard ServiceNow form for the incident table.
- Client Scripts: JavaScript that runs on form load, field change and submit.
- UI Policies: Condition-based rules that make fields mandatory, visible or read-only.
- IT Support Team: Receives complete, validated incidents.

Platform

ServiceNow: Provides the environment for the Incident form, script and policy configuration, and data storage.

Client Scripts and UI Policies

Both are configured under the Incident table and use the `g_form` client-side API.

- Client Script types: `onLoad`, `onChange`, `onSubmit`.
- UI Policy components: Conditions and UI Policy Actions.
- Order and Global/Run-scripts settings control execution.

Database

The project uses the ServiceNow platform's built-in database.

Table / Component	Purpose
Incident (incident)	Stores incident details such as short description, category, priority and state.
Client Script (sys_script_client)	Stores the Client Script definitions.
UI Policy (sys_ui_policy)	Stores UI Policy definitions.
UI Policy Action (sys_ui_policy_action)	Stores the field actions of each UI Policy.

The project uses existing ServiceNow tables and does not require a separate database.

4. Setup Instructions

Prerequisites

- A ServiceNow Developer account and instance.

- A modern web browser such as Google Chrome or Microsoft Edge.
- Administrator or developer permissions to create Client Scripts and UI Policies.
- Basic knowledge of ServiceNow Incident Management and JavaScript.
- Internet connectivity.

Installation and Configuration

Step 1: Access ServiceNow Developer Instance

Sign in to the ServiceNow Developer Portal and open your assigned instance.

Step 2: Create the Client Scripts

- Open the Application Navigator and go to System Definition > Client Scripts.
- Click New and select the Incident table.
- Choose the Type (onLoad, onChange or onSubmit) and, for onChange, the Field name.
- Enter the script, check Active and click Submit.

Step 3: Create the UI Policies

- Go to System UI > UI Policies and click New.
- Select the Incident table and set the Short description.
- Define the Conditions (for example State is Resolved).
- Save, then add UI Policy Actions to set fields Mandatory, Visible or Read-only.

Step 4: Sample Client Script

```
function onSubmit()
{
  var sd = g_form.getValue('short_description');
  if (sd.length < 10)
  {
    g_form.addErrorMessage('Short description must be at least 10 characters.');
```

Step 5: Test the Configuration

- Open a new Incident form and confirm the onLoad message.
- Change Priority to 1 - Critical and confirm the alert.
- Submit with a short description under 10 characters and confirm the error.
- Set State to Resolved and confirm Close notes and Close code are mandatory.

5. Folder Structure

Unlike a traditional application, this project does not contain separate client and server folders. The implementation is configured within the ServiceNow platform, and the GitHub repository holds the documentation.

Logical Project Structure

```
Client Script & UI Policy (Incident) - ServiceNow/
|
|-- 1. Ideation Phase/
|   |-- Empathy_Map_Canvas.pdf
|   |-- Define_The_Problem_Statement.pdf
|   `-- Brainstorm___Idea_Prioritization.pdf
|
|-- 2. Requirement Analysis/
|   |-- Customer Journey Map.pdf
|   |-- Solution Requirements.pdf
|   `-- Technology Stack.pdf
|
|-- 3. Project Design Phase/
|   |-- Problem - Solution Fit.pdf
|   |-- Proposed Solution.pdf
|   `-- Solution Architecture.pdf
|
|-- 4. Project Planning Phase/
|   `-- Project Planning.pdf
|
|-- 5. Project Development Phase/
|   |-- Performance Testing/
|   |   |-- Functional_Performance_Testing.pdf
|   |   |-- Model_Performance_Test_Summary.pdf
|   |   |-- Model_Performance_Test_Metrics.pdf
|   |   `-- Model_Performance_Test_Dashboard.pdf
|   `-- User Acceptance Testing/
|       `-- User_Acceptance_Testing.pdf
|
|-- 6. Project Documentation/
|   |-- Project_Documentation.pdf
|   `-- Project_Report.pdf
|
|-- README.md
```

This is a suggested GitHub documentation structure. The actual Client Scripts and UI Policies are maintained in the ServiceNow instance.

6. Running the Application

Since this project is implemented on the ServiceNow platform, there are no local servers to start.

Steps to Run the Project

1. Log in to your ServiceNow Developer Instance.
2. Open System Definition > Client Scripts and confirm the scripts are Active.
3. Open System UI > UI Policies and confirm the policies are Active.
4. Navigate to Incident > Create New.

5. Observe the onLoad message.
6. Change Priority and State and observe the alerts and field behaviour.
7. Enter a short description and submit the incident.
8. Verify validation messages and the saved record.

Expected Output

The form should display messages, enforce mandatory and read-only fields and block invalid submissions as configured.

7. API Documentation

API Integration

The project uses ServiceNow's built-in client-side APIs. It does not require a separately developed REST API or external backend service.

Core Operations

Operation	Description	API / Method
Read a field value	Gets the current value of a form field	<code>g_form.getValue()</code>
Set a field value	Updates a field value	<code>g_form.setValue()</code>
Show information message	Displays a message at the top of the form	<code>g_form.addInfoMessage()</code>
Show error message	Displays an error message	<code>g_form.addErrorMessage()</code>
Make field mandatory	Sets a field as required	<code>g_form.setMandatory()</code> / UI Policy Action
Make field read-only	Prevents editing of a field	<code>g_form.setReadOnly()</code> / UI Policy Action
Stop submission	Cancels form submit	return false in onSubmit

Optional REST API Integration

ServiceNow REST APIs can be used for future integration. External API integration is not required for the current implementation.

8. Authentication

The project uses ServiceNow's built-in authentication and access control mechanisms.

Authentication

- Users access the instance using authorized login credentials.

- Authentication is managed by the ServiceNow platform.

Authorization

- Administrator: Can configure Client Scripts, UI Policies and manage users.
- Developer: Can create and maintain scripts and policies, subject to assigned permissions.
- IT Support User: Can open and update incidents according to permissions.
- End User: Can create and view incidents based on available access rights.

Security

The project relies on ServiceNow access control rules (ACLs) to protect records. Client Scripts improve user experience but are not a security control, because they run in the browser. Sensitive rules should also be enforced on the server.

9. User Interface

The project uses the standard ServiceNow interface for incident creation, script and policy configuration, and testing.

Main Interface Components

1. ServiceNow Home: Provides access to modules and applications.
2. Incident Form: Where users create incidents and where scripts and policies act.
3. Client Script Form: Used to define script type, table, field and code.
4. UI Policy Form: Used to define conditions and policy actions.
5. Messages and Mandatory Indicators: Show the results of scripts and policies on the form.

Screenshots

The screenshot shows the ServiceNow Developer Instance interface for creating a new incident. The browser address bar displays the URL: https://instance.service-now.com/incident.do?sys_id=-1. The page header includes the ServiceNow logo and navigation icons. A green notification bar at the top states: "INFO: A Priority 1 (Critical) incident has been created." Below this, the "Create New" form is visible. The form includes the following fields and controls:

- Number:** A text input field containing "INC10011".
- Urgency:** A dropdown menu set to "1 - High".
- Status:** A dropdown menu set to "1 - High".
- Configuration Item:** A text input field with a search icon and a help icon.
- Priority:** A dropdown menu set to "1 - Critical", which is highlighted with a red border.
- End Faree:** A text input field.
- Short Description:** A text input field.
- Description:** A large text area for the incident description.

At the bottom of the form, there are two buttons: "Submit" and "Resolve".

Figure 1: ServiceNow Developer Instance

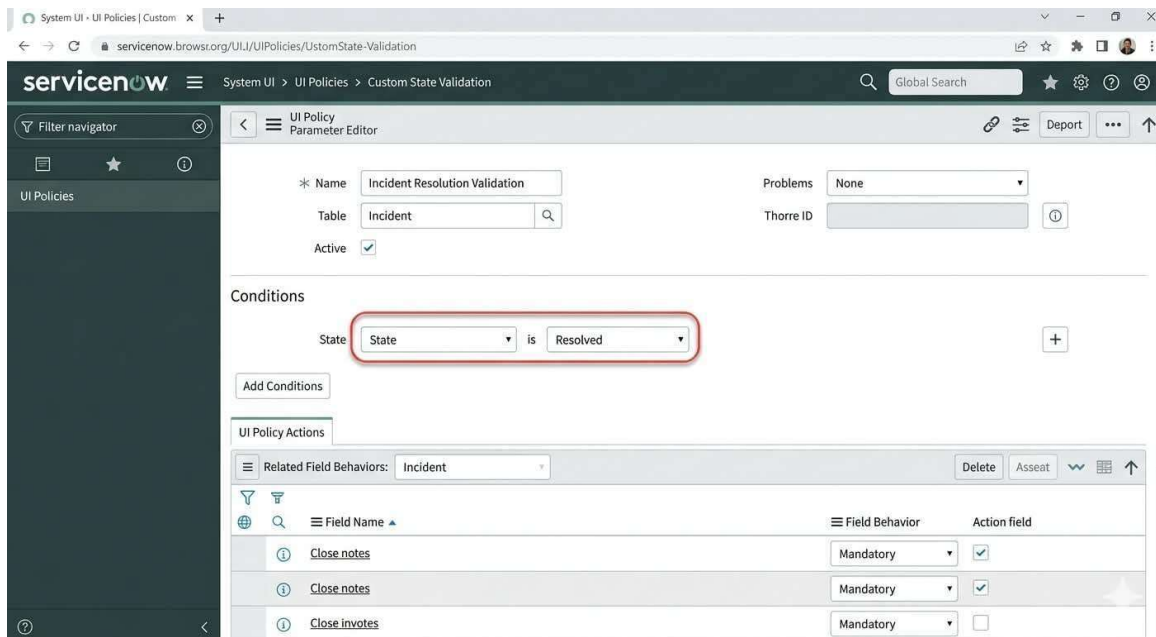


Figure 2: Client Script

10. Testing

Testing was performed to verify the functionality, reliability and performance of the implemented Client Scripts and UI Policies.

Testing Types

Functional Testing

- onLoad message validation.
- onChange alert on Priority.
- onSubmit short description validation.
- UI Policy mandatory fields (Resolved).
- UI Policy conditional and read-only behaviour.

Performance Testing

- Form load time.
- Client Script response time.
- Repeated execution reliability.

User Acceptance Testing (UAT)

The implementation was tested to confirm it meets the needs of end users and IT support staff.

Test Results

Testing Type	Total Test Cases	Passed	Failed
Functional Testing	5	5	0
Performance Testing	3	3	0

Total	8	8	0
-------	---	---	---

100% Test Case Pass Rate. All 8 test cases were completed successfully. The tester was V. Sriram.

11. Screenshots or Demo

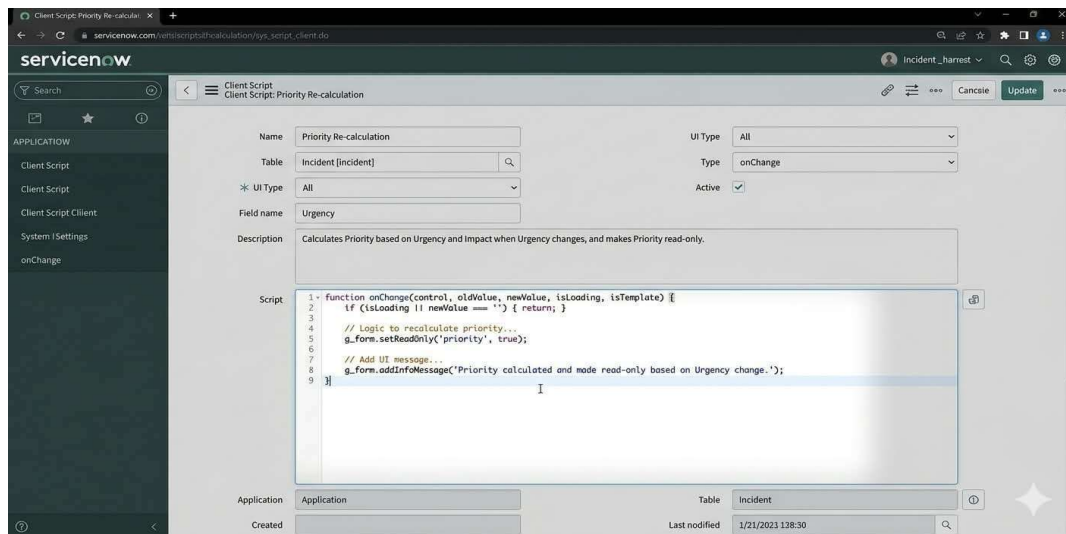


Figure 5: Test Results

Demonstration Steps

1. Open the ServiceNow Developer Instance.
2. Open a new Incident form.
3. Observe the onLoad message.
4. Change Priority to 1 - Critical and observe the alert.
5. Submit with a short description and observe the error.
6. Set State to Resolved and observe the mandatory fields.
7. Set State to Closed and observe the read-only fields.

Demo Video

https://drive.google.com/file/d/1Q_C4j7-QYx7Cwkiy8gge99KCQe3UFC/view?usp=drivesdk

GitHub Repository

<https://github.com/srinandhabala0666/Implement-Client-Script-UI-Policy-Incident->

12. Known Issues

During testing, no major functional issues were identified. The following considerations apply:

- Client Scripts run only in the browser and do not validate data from imports or APIs.
- UI Policy and Client Script behaviour depends on execution order and conditions.
- Changes to field names or choice values require script updates.
- Many scripts on a single form can affect load time.

- Behaviour depends on the availability and configuration of the instance.

These are considerations for maintenance and extension rather than confirmed defects.

13. Future Enhancements

1. Server-side Enforcement: Add Business Rules or Data Policies for all channels.
2. More Validation Rules: Extend checks to additional fields.
3. Catalog Client Scripts: Apply similar logic to Service Catalog items.
4. Flow Designer Integration: Automate classification and assignment.
5. Notifications: Alert users and teams on key changes.
6. Reporting: Build reports on data quality and incident trends.
7. Continuous Improvement: Refine rules using user feedback.

Conclusion

The Implement Client Script & UI Policy (Incident) project demonstrates how client-side logic in ServiceNow can improve the quality of incident data and guide users as they work.

By using Client Scripts and UI Policies, the Incident form validates input, shows helpful messages and controls field behaviour based on conditions.

The implementation was tested using functional, performance and user acceptance scenarios, with all test cases passing. It provides a foundation that can be extended with server-side enforcement, Flow Designer automation and reporting.